

```
# -*- coding: utf-8 -*-
```

```
"""
```

```
Created on Sun Aug 16 10:03:04 2026
```

```
@author: Ashutosh Tiwari
```

```
"""
```

```
# AI Prompt Optimizer Bot (NLP & Prompt Engineering)
```

```
# Team: 1st Year B.Tech (AI & Machine Learning)
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
import numpy as np
```

```
print("""65)
```

```
print(" 🤖 Welcome to AI Prompt Optimizer & Assistant Bot ")
```

```
print("""65)
```

```
print("Turn your simple questions into 'Pro AI Prompts' using NLP!\n")
```

```
# The AI's Knowledge Base (Simple Prompts vs Engineered Mega-Prompts)
```

```
database_prompts = [
```

```
    "tell me about cricket match or sports",
```

```
    "make a fitness routine or running schedule",
```

```
    "how to write python code or programming",
```

```
    "tell me a story or write something creative"
```

```
]
```

```
engineered_prompts = [
```

```
    "[Act as a Professional Sports Analyst] Analyze the latest match strategies, focusing on player roles, pitch conditions, and match-winning tactics.
```

```
Provide statistical examples.",
```

```
    "[Act as a Certified Fitness Coach] Create a detailed morning running schedule and diet plan tailored for a beginner aiming to improve stamina and track their distance goals.",
```

```
    "[Act as a Senior Software Engineer] Write clean, bug-free Python code for this problem. Include line-by-line comments, optimize the logic, and add error-handling.",
```

```
    "[Act as a Creative Writer] Write an engaging, suspenseful short story with vivid descriptions, strong character development, and a sudden plot twist at the end."
```

```
]
```

```
# -----
```

```
# Machine Learning Logic (NLP - Natural Language Processing)
```

```
# -----
```

```
# Converting text to mathematical vectors (Because ML models don't understand English)
```

```
vectorizer = TfidfVectorizer()
```

```
vector_database = vectorizer.fit_transform(database_prompts)
```

```
def optimize_prompt(user_input):
```

```
    # Convert user's input into the same numerical format
```

```
    user_vector = vectorizer.transform([user_input])
```

```
    # Calculate Cosine Similarity (Checking context matching)
```

```
    similarity_scores = cosine_similarity(user_vector, vector_database)
```

```
    # Finding the best matching category
```

```
    best_match_index = np.argmax(similarity_scores)
```

```
    highest_score = similarity_scores[0][best_match_index]
```

```
    # If the bot understands the context even a little bit
```

```
    if highest_score > 0.1:
```

```

        return engineered_prompts[best_match_index]
    else:
        # Default Universal Prompt if the topic is totally new
        return f"[Act as a Top-Tier Expert] I want you to provide a highly detailed, step-by-step, and logically structured answer to the following request:
        {user_input}"

# -----
# Chatbot Loop (Interface)
# -----
while True:
    print("\n" + "-"*40)
    user_text = input("👤 Enter your simple question (or type 'exit' to stop): ").lower()

    if user_text == 'exit':
        print("🛑 Bot shutting down. Happy Prompting!")
        break

    print("\n[Bot is applying NLP & Prompt Engineering...]")

    # Calling the ML Function
    pro_prompt = optimize_prompt(user_text)

    print("\n✨ Your Optimized Mega-Prompt:")
    print(">>", pro_prompt)
    print("\n💡 (Copy and paste this into ChatGPT or Gemini for an amazing result!)")

```